Stats on a FILE with Read

```c
#include <stdio.h>
#include <string.h>
#define NB_DEMANDES 3
#define TAILLE 256
#define start 32
#define stop 253
typedef unsigned long int Statistique[TAILLE];
FILE* demander_fichier(void);
void initialise_statistique(Statistique a_initialiser);
unsigned long int collecte_statistique(Statistique a_remplir,
FILE* fichier_a_lire);
void affiche(Statistique a_afficher, unsigned long int total,
unsigned short int taille);
int main(void){
  FILE* fichier = demander_fichier();
  if (fichier == NULL) {
    printf("=> j'abandonne !\n");
    return 1;
  } else {
    Statistique stat;
    initialise_statistique(stat);
    affiche(stat, collecte_statistique(stat, fichier), stop - start + 1);
    fclose(fichier);
  }
  return 0;
}
```

```c
FILE* demander_fichier(void){
  FILE* f = NULL;
  char nom_fichier[FILENAME_MAX+1];
  size_t taille_lue = 0;
  unsigned short int nb = 0;
  do {
    ++nb;
    do {
      printf("Nom du fichier `a lire : "); fflush(stdout);
      fgets(nom_fichier, FILENAME_MAX+1, stdin);
      taille_lue = strlen(nom_fichier);
      if ((taille_lue >= 1) && (nom_fichier[taille_lue-1] == '\n'))
      nom_fichier[--taille_lue] = '\0';
    } while ((taille_lue == 0) && !feof(stdin));
    if (nom_fichier[0] == '\0') {
      return NULL;
    }
    f = fopen(nom_fichier, "r");
    if (f == NULL) {
      printf("-> ERREUR, je ne peux pas lire le fichier \"%s\"\n",
nom_fichier);
    } else {
      printf("-> OK, fichier \"%s\" ouvert pour lecture.", nom_fichier);
    }
  } while ((f == NULL) && (nb < NB_DEMANDES));
  return f;
  }

void initialise_statistique(Statistique stat){
  int i;
  for (i = 0; i < TAILLE; ++i) {
    stat[i] = 0;
  }
}

unsigned long int collecte_statistique(Statistique stat, FILE* f){
  int c;
  unsigned long int nb = 0;
  while ((c = getc(f)) != EOF) {
    if (( ((unsigned char) c) >= start) && ( ((unsigned char) c) <=
stop ) ) {
      ++(stat[c - start]);
      ++nb;
      }
  }
  return nb;
}


void affiche(Statistique stat, unsigned long int nb, unsigned short int
taille) {
  unsigned short int i;
  if (nb == 0) {
    for (i = 0; i < taille; ++i)
    nb += stat[i];
  }
  printf("STATISTIQUES :\n");
  for (i = 0; i < taille; ++i) {
    if (stat[i] != 0) {
      printf("%c : %10lu - %6.2f%%\n", (char) (i+start), stat[i], 100.0 *
stat[i] / (double) nb);
    }
  }
}
```

Vector

```c
#include <stdlib.h>
#include <stdint.h>

typedef int type_el;

typedef struct {
  size_t size; // nb d'elem utilisees
  size_t allocated; // capacitee allouee
  type_el* content; // zone allouee dynamiquement
} vector;

#define VECTOR_PADDING 128

#ifndef SIZE_MAX
#define SIZE_MAX (~(size_t)0)
#endif

vector* vector_construct(vector* v) {
  if (v != NULL) {
    vector result = (vector){0,0,NULL};
    result.content = calloc(VECTOR_PADDING,sizeof(type_el));
    if (result.content == NULL) return NULL;
    result.allocated = VECTOR_PADDING;
    *v = result;
  }
  return v;
}

void vector_delete(vector* v) {
  if (v != NULL && v->content != NULL) {
    free(v->content);
    v->content = NULL;
    v->size = 0;
    v->allocated = 0;
  }
}

vector* vector_enlarge(vector* v) {
  if (v != NULL) {
    vector result = *v;
    result.allocated += VECTOR_PADDING;

    if (result.allocated > SIZE_MAX/sizeof(type_el)) return NULL;

    type_el* temp =
      realloc(result.content,result.allocated*sizeof(type_el));
    if (temp == NULL) return NULL; // v inchangé

  result.content = temp; // si succes de reall. on maj le pointeur
  *v = result;
  }
  return v;
}

size_t vector_push(vector* v,type_el val) {
  if (v != NULL) {
    while (v->size >= v->allocated) {
      if (vector_enlarge(v) == NULL) return 0;
    }
    v->content[v->size] = val;
    ++(v->size);
    return v->size;
  }
  return 0;
}
```

Allocation tableau dynamique

```c
1D
t_type* arr = calloc(n, sizeof(t_type));
if(arr == NULL)return NULL;
free(arr);

2D
double** m = calloc(rows, sizeof(double*));
if(m==NULL)return NULL;
for(size_t l = 0; l < rows; l++){
  m[l] = calloc(cols, sizeof(double));
  if(m[l]==NULL){ // on supprime les lignes deja allouees
    for(int t = (int)l; t>=0;--t){
      free(m[t]);
      m[t] = NULL;
    }
    free(m); m = NULL; return NULL
  }
}
//Libération ordre inverse
for(size_t i = 0; i < rows; i++){
  free(m[i]);
  m[i] = NULL;
}
free(m);
m = NULL;

<- Alloc, d'une struct contenant tableau dyn
```

Realloc safe

```c
// toujours passer par un temporaire
t_type* temp = realloc(arr- >elements, new_size * sizeof(t_type));
if (temp == NULL) {
  // arr->elements est toujours valide ici ! (abandon)
  free(arr->elements);
  free(arr);
  return -1;
}
arr->elements = temp; // seulement si succès
//  NE PAS free(temp) après arr->elements = temp :
// les deux pointeurs pointent vers le même bloc !
```

Write to a file

```c
#include <stdio.h>
#include <string.h>

/* taille maximale pour un nom */
#define TAILLE_NOM 1024

int main(void)
{
    char const nom_fichier[] = "data.dat"; /* le nom du fichier */
    FILE* sortie;
    int taille_lue;

    char nom[TAILLE_NOM+1]; /* pour stocker le "nom" à lire depuis le
          clavier */
    unsigned int age; /* pour stocker l'"age" à lire depuis le clavier
         */

    /* Ouverture de data.dat en écriture (w=write) */
    sortie = fopen(nom_fichier, "w");

    /* on teste si l'ouverture du flot s'est bien réalisée */
    if (sortie == NULL) {
        fprintf(stderr,
                "Erreur: le fichier %s ne peut etre ouvert en écriture
                     !\n",
                nom_fichier);
        return 1; /* retourne un autre chiffre que 0 car il y a eu une
             erreur */
    }

    /* itération sur les demandes à entrer :
       on continue tant que stdin est lisible */
    while (!feof(stdin)) {

        /* tant qu'un nom vide est entré */
        do {
            printf("Entrez un nom (CTRL+D pour terminer) : ");
            fflush(stdout);

            fgets(nom, TAILLE_NOM+1, stdin);

            taille_lue = strlen(nom) - 1;
            if ((taille_lue >= 0) && (nom[taille_lue] == '\n'))
                nom[taille_lue] = '\0';

        } while (!feof(stdin) && (taille_lue < 1));

        if (!feof(stdin)) {

            /* L'utilisateur a bien saisi un nom,
               on peut donc lui demander de saisir l'age. */
            printf("âge : ");
            fflush(stdout);

            taille_lue = scanf("%u", &age);

            if (taille_lue != 1) {
                printf("Je vous demande un age (nombre entier positif)
                     pas "
                       "n'importe quoi !\nCet enregistrement est annulé.\
                            n");

                while (getc(stdin) != '\n'); /* vide le tampon d'entrée
                     */
            }
            else {
                getc(stdin); /* récupère le \n résiduel */

                /* écriture dans le fichier */
                fprintf(sortie, "%s %d\n", nom, age);
            }
        }
    }

    /* purisme : retour à la ligne pour finir proprement la question */
    putchar('\n');

    fclose(sortie); /* fermeture du fichier */

    return 0;
}
```

Linked Lists

```c
#include <stdio.h>
#include <stdlib.h>

#define LISTE_VIDE NULL
#define est_vide(L) ((L) == LISTE_VIDE)

#define type_el int
#define affiche_el(T) printf("%d", (T))

typedef struct Element Element;
typedef Element* ListeChainee;

struct Element {
    type_el valeur;
    ListeChainee suite;
};

void insere_entete(ListeChainee* liste, type_el val) {
    ListeChainee tmp = *liste;

    *liste = malloc(sizeof(Element));
    if (*liste != NULL) {
        (*liste)->valeur = val;
        (*liste)->suite = tmp;
    } else {
        *liste = tmp;
    }
}

void insere_apres(Element* e, type_el val) {
    Element* n = malloc(sizeof(Element));
    if (n != NULL) {
        n->valeur = val;
        n->suite = e->suite;
        e->suite = n;
    }
}

void supprime_tete(ListeChainee* liste) {
    if (!est_vide(*liste)) {
        ListeChainee tmp = (*liste)->suite;
        free(*liste);
        *liste = tmp;
    }
}

void supprime_suivant(Element* e) {
    supprime_tete(&(e->suite));
}

size_t taille(ListeChainee liste) {
    size_t t = 0;
    while (!est_vide(liste)) {
        ++t;
        liste = liste->suite;
    }
    return t;
}

void affiche_liste(ListeChainee liste) {
    ListeChainee i;
    putchar('(');
    for (i = liste; !est_vide(i); i = i->suite) {
        affiche_el(i->valeur);
        if (!est_vide(i->suite)) printf(", ");
    }
    putchar(')');
}
```

Ecriture dans fichier (scanf)

```c
int process(const char* filename) {
  FILE* file = fopen(filename,"w");
  if (file == NULL) {
    fprintf(stderr, "Impossible d'ouvrir %s\n", filename);
    return -1;
  }
  char name[100];
  int age;
  printf("Enter a name (CTRL+D to finish): ");
  while (scanf("%99s", name) != EOF) {
    printf("age: ");
    if (scanf("%d", &age) != 1 || age < 0) {
      printf("(positive integer)!\n");
      printf("canceled.\n");
      int c; // vider la ligne courante du buffer
      while ((c = getchar()) != '\n' && c != EOF);
    } else {
      fprintf(file, "%s %d\n", name, age);
    }
    printf("Entrer a nane (CTRL+D to finish): ");
  }
  fclose(file);
  return 0;
}
```